

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Реализация алгоритма Кнута-Морриса-Пратта

Студент гр. 4345

Беденков А. В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить теоретические основы алгоритма Кнута-Морриса-Пратта (КМП) и применить его на практике для решения задач точного поиска подстроки и проверки циклической эквивалентности строк.

Постановка задачи

Пункт 1.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 500000$) найдите все вхождения P в T .

Ввод: строка P (шаблон), строка T (текст).

Вывод: индексы начал вхождений P в T , разделенные запятой; если P не входит в T , то вывести -1.

Тестовые данные:

Ввод

ab

abab

Вывод

0,2

Пункт 2.

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход: первая строка A , вторая строка B

Выход: если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе -1. Если возможно несколько сдвигов, вывести первый индекс.

Тестовые данные:

Ввод

defabc

abcdef

Вывод

3

Основные теоретические положения

Алгоритм КМП основывается на предварительном анализе структуры шаблона для исключения повторных сравнений уже проверенных символов.

- Префикс-функция: Это массив pi , где $pi[i]$ — длина самого длинного собственного префикса подстроки $P[0..i]$, который одновременно является её суффиксом.
- Идея пропуска символов: Если при сопоставлении шаблона с текстом происходит несовпадение на позиции q , мы знаем, что часть шаблона до этой позиции уже совпала с текстом. Массив pi позволяет "сдвинуть" шаблон сразу к следующему возможному совпадению, минуя заведомо ложные позиции.
- Линейность: Указатель в основном тексте никогда не возвращается назад, что обеспечивает высокую скорость работы на больших объемах данных.

Выполнение работы

В рамках лабораторной работы алгоритм Кнута-Морриса-Пратта реализован в виде двух основных функций, каждая из которых оптимизирована под свою задачу. Для корректной работы с потоковым вводом в тестирующих системах чтение данных обернуто в блок `try...except EOFError`.

Функция `solve_kmp()` (Поиск всех вхождений шаблона в текст):

Генерация префикс-функции: Алгоритм создает массив pi длиной, равной длине шаблона p . В цикле переменная j отслеживает длину совпадающего префикса и суффикса. При несовпадении j откатывается назад ($j = pi[j - 1]$), что позволяет эффективно найти самую длинную повторяющуюся часть.

Поиск по тексту: Выполняется один линейный проход по строке t . Указатель j теперь отвечает за количество совпавших символов шаблона.

Обработка совпадений и наложений: Как только j становится равно длине шаблона, вычисляется стартовый индекс $(i - len(p) + 1)$ и добавляется в массив `res`. Сразу после этого j принудительно сдвигается по префикс-функции ($j = pi[j - 1]$). Это позволяет алгоритму продолжить поиск и найти вхождения, которые могут накладываться друг на друга (например, при поиске `aba` в `ababa`). В конце результаты выводятся через запятую (или `-1`, если массив пуст).

Функция `solve_cyclic_shift()` (Проверка циклического сдвига):

Предварительные проверки: Функция сразу отсекает заведомо ложные случаи: если длины строк a и b не равны, выводится `-1`. Если строки пустые — выводится `0`.

Виртуальное зацикливание: Чтобы проверить, является ли строка a сдвигом b , необходимо искать b внутри удвоенной строки $a + a$. Для экономии памяти физического копирования не происходит. Вместо этого

цикл работает до $2 * n - 1$, а нужный символ строки извлекается с помощью операции взятия остатка от деления: $a[i \% n]$.

Ранний выход: Поскольку по условию задачи требуется найти лишь первый возможный сдвиг, при первом же полном совпадении ($j == m$) функция сразу выводит индекс и завершает свою работу оператором `return`, экономя процессорное время.

Оценка сложности алгоритма

Временная сложность:

- Общая сложность: $O(N + M)$, где N — длина текста (или строки a), а M — длина шаблона (или строки b).
- Построение префикс-функции: Требуется $O(M)$ времени, так как количество уменьшений счетчика j во внутреннем цикле `while` не может превышать общего количества его увеличений во внешнем `for`.
- Основной поиск: Требуется $O(N)$ для первой задачи и $O(2N)$ для второй (что асимптотически равно $O(N)$). Указатель по тексту движется строго вперед без возвратов.

Сложность по памяти:

- Общая сложность: $O(N + M)$ или $O(M)$ дополнительной памяти.
- Основная память уходит на хранение двух введенных строк $O(N + M)$.
- Дополнительная память выделяется только под массив префикс-функции pi , который занимает $O(M)$ памяти.
- Функция `solve_cyclic_shift` реализована `in-place` (без выделения памяти под удвоенную строку), что делает её максимально эффективной с точки зрения потребления ресурсов.

Тестирование

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование solve_kmp

№	Входные данные	Выходные данные
1	ab abab	0,2
2	a abababaab	0,2,4,6,7
3	abacaba abacabacaba	0,4
4	aba	-1

Таблица 2 – Тестирование solve_cyclic_shift

№	Входные данные	Выходные данные
1	aaaaa aaaaa	0
2	abccba cbaabc	3
3	aboba obaab	2
4	aboba obaab	-1

Выводы

В ходе работы был реализован алгоритм КМП, обеспечивающий эффективный поиск подстрок за линейное время. Использование префикс-функции позволило оптимизировать процесс сопоставления, а механизм виртуального заикливания индекса — решить задачу циклического сдвига с минимальными затратами памяти.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

solve_kmp.py

```
def solve_kmp():
    try:
        p = input()
        t = input()
    except EOFError:
        return

    if not p:
        print("-1")
        return

    m = len(p)
    n = len(t)

    print(f"\n[ЭТАП 1] Вычисляем префикс-функцию для шаблона P = '{p}'")
    pi = [0] * m
    j = 0
    for i in range(1, m):
        print(f"Символ P[{i}] = '{p[i]}', сравниваем с P[{j}] = '{p[j]}'")
        while j > 0 and p[i] != p[j]:
            j = pi[j - 1]
            print(f"Не совпало. Откат указателя j по pi-массиву: j = {j}")
        if p[i] == p[j]:
            j += 1
            print(f"Совпало! Увеличиваем j до {j}")
        pi[i] = j
        print(f"Записываем pi[{i}] = {j}")
    print(f"Итоговый pi-массив: {pi}\n")

    print(f"[ЭТАП 2] Начинаем поиск шаблона P в тексте T = '{t}'")
    res = []
    j = 0
    for i in range(n):
        print(f"Шаг i={i}: Текст T[{i}] = '{t[i]}', Шаблон P[{j}] = '{p[j]}'")
        if j < m else 'вне границ')
        while j > 0 and t[i] != p[j]:
            j = pi[j - 1]
            print(f"Не совпало. Откат j по pi-массиву: j = {j}")
        if t[i] == p[j]:
            j += 1
            print(f"Совпало! Двигаемся дальше по шаблону, j = {j}")
        if j == m:
            start_index = i - m + 1
            print(f"НАЙДЕНО ПОЛНОЕ ВХОЖДЕНИЕ! Индекс начала: {start_index}")
            res.append(str(start_index))
            j = pi[j - 1]
            print(f"Откат j для поиска возможных наложений: j = {j}")
```

```

        print("\n[РЕЗУЛЬТАТ]")
        print(", ".join(res) if res else "-1")

if __name__ == '__main__':
    solve_kmp()

```

solve_cyclic_shift.py

```

def solve_cyclic_shift():
    try:
        a = input()
        b = input()
    except EOFError:
        return

    n = len(a)
    m = len(b)

    print(f"\n[СТАРТ] Проверка циклического сдвига: A = '{a}', B = '{b}'")
    if n != m:
        print("Длины строк не равны. Они не могут быть сдвигом друг друга.")
        print("\n[РЕЗУЛЬТАТ]\n-1")
        return
    if n == 0:
        print("\n[РЕЗУЛЬТАТ]\n0")
        return

    print(f"\n[ЭТАП 1] Вычисляем префикс-функцию для шаблона B = '{b}'")
    pi = [0] * m
    j = 0
    for i in range(1, m):
        print(f"Символ B[{i}] = '{b[i]}', сравниваем с B[{j}] = '{b[j]}'")
        while j > 0 and b[i] != b[j]:
            j = pi[j - 1]
            print(f"Не совпало. Откат j: j = {j}")
        if b[i] == b[j]:
            j += 1
            print(f"Совпало! j = {j}")
        pi[i] = j
    print(f"Итоговый pi-массив для B: {pi}\n")

    print(f"[ЭТАП 2] Ищем B в зацикленной строке A")
    j = 0
    for i in range(2 * n - 1):
        char_a = a[i % n]
        print(f"Шаг {i}: Виртуальный символ A[{i} % {n}] = '{char_a}', Шаблон B[{j}] = '{b[j] if j < m else 'вне границ'}'")

        while j > 0 and char_a != b[j]:
            j = pi[j - 1]
            print(f"Не совпало. Откат j: j = {j}")
        if char_a == b[j]:
            j += 1

```

```
        print(f" Совпало! Увеличиваем j до {j}")

    if j == m:
        ans = i - m + 1
        print(f" НАЙДЕНО! Строка В найдена внутри сдвоенной A!")
        print(f"\n[РЕЗУЛЬТАТ]")
        print(ans)
        return

    print("\n[РЕЗУЛЬТАТ]")
    print("-1")

if __name__ == '__main__':
    solve_cyclic_shift()
```